Structure vs. Semantics in Code-to-Architecture Mapping: The Role of the Dependency Graph

Rakhshanda Jabeen*†, Morgan Ericsson†, Jonas Nordqvist†, and Anna Wingkvist†
*Electrolux Professional AB, Ljungby, Sweden
†Linnaeus University, Växjö, Sweden
Email: {rakhshanda.jabeen | morgan.ericsson | jonas.nordqvist | anna.wingkvist}@lnu.se

***Abstract*—Architectural technical debt accrues whenever a system's implementation drifts from its intended decomposition into modules, and the debt compounds unnoticed until the mapping between source code entities and architecture is recovered. Such mappings are often missing, incomplete, or too costly to maintain manually. We study this mapping as semi-supervised node classification over a directed multi-relational dependency graph, with files as nodes, typed dependency relations as edges, and node features encoding code identifiers and directory layout. Starting from a small labeled sample, an iterative self-training loop expands the mapping to the full system. We compare a graph convolution network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP) against a Naive Bayes reference, holding features and training procedure fixed across the three learned models so that any difference between them is attributable to the dependency graph. All methods are evaluated on ten open-source systems spanning Java and C/C++, across 200 runs per system. Every learned model exceeds the reference baseline by a wide margin, and the GAT attains the highest F1 score. The MLP remains competitive on small to medium-sized systems, but the GNN variants outperform it by a wide margin on Chrome, the largest and most complex system in the benchmark. The graph changes the character of the mapping and not only its accuracy: the GNN variants are more precise and noticeably more repeatable across initial label draws, while the MLP achieves higher recall. Files that remain persistently classified wrong either sit in a neighborhood dominated by another module or have almost no neighborhood at all. Among the design choices tested, modeling dependency types as separate relations rather than a single merged edge type contributes the largest gain.**

I. Introduction

Software architecture defines a system's high-level structure and the dependencies among its principal elements, and it provides the basis for reasoning about maintainability, security, and other quality attributes [1]. That reasoning requires the implementation to stay aligned with the intended design. When it does not, the gap is architectural technical debt [2]: cheap to introduce, but diffuse and relational rather than local to a single file, and therefore expensive to detect and repay once it accumulates. Large language models (LLMs) are now widely used to generate or assist with code commits, compounding this problem because they produce code that satisfies the immediate prompt without visibility into the system's module structure [3]. Studies of AI-generated code report non-trivial clone-like repetition across tasks, suggesting that these models reproduce learned patterns rather than reusing project-specific abstractions [4]. As this code enters the system unchecked, the implementation drifts from the intended design, causing architectural erosion [5].

Detecting this drift requires comparing the intended architecture against the implemented one. Static Architecture Conformance Checking (SACC) addresses this comparison, and reflexion modeling is a SACC technique that classifies relations between the intended and implemented architectures as convergent, divergent, or absent [6, 7]. However, reflexion modeling depends on a mapping between source code entities and architectural modules, and such mappings are often missing, incomplete, or expensive to maintain manually [8]. Even for medium-sized systems, recovering or maintaining the architectural decomposition can require substantial expert effort [9]. Automated code-to-architecture (C2A) mapping is therefore an important prerequisite for practical conformance checking.

Prior work on C2A mapping has explored supervised and recommendation-based approaches. Supervised approaches compute *attraction values* that quantify the association strength between source code entities and architectural modules [10, 11]. Olsson et al. [12] proposed a semi-supervised Naive Bayes mapper that combines syntactic and semantic information to assign source code entities to architectural modules. InMap [13] took a recommendation-based view instead, using information retrieval over the codebase to suggest candidate mappings with minimal architectural documentation and no training data.

A separate line of work targets architecture recovery, where the decomposition itself must be discovered rather than assumed. Recent recovery approaches include graph neural networks (GNNs) combined with contrastive learning and autoencoders [14, 15], along with language models applied directly to source code for recovering design structures [16], for agent-based recovery of large legacy systems [17], and for extracting architectural entities to support traceability [18]. The distinction from C2A mapping matters here: the module set is unknown in recovery, whereas in mapping it is given by the intended architecture and the task is to assign each file to one of those modules. It also matters for evaluation, since many recovery benchmarks are open source, and a decomposition reproduced by a pretrained model may reflect code understanding, prior exposure to the system, or both.

Mapping methods that learn from a system's own structure and can be evaluated against a known ground truth therefore remain important.

In this research, we study C2A mapping[1] as semi-supervised node classification over a multi-relational directed dependency graph. Source files are nodes, typed static dependencies between them are edges, and node features encode code identifiers via Word2Vec embeddings together with directory context. A small set of labeled files is expanded into a complete mapping through iterative self-training. We compare three encoders under the same features and training procedure: a graph convolution network (GCN), a graph attention network (GAT), and a graph-free multi-layer perceptron (MLP). Holding the feature representation, the classifier head, and the self-training loop fixed isolates what the dependency graph contributes beyond the node features alone.

We evaluate the approach on ten open-source systems spanning Java and C/C++, including the industrial-scale Chrome codebase, and report both mapping accuracy and computational cost. This paper extends our earlier study [19] along five lines: a broader benchmark that adds C/C++ systems; Word2Vec embeddings in place of the earlier bag-of-words encoding for code identifiers; GAT, added to assess the impact of attention-based aggregation on the mapping problem; an MLP baseline, added to isolate the graph's contribution; and a corrected, substantially revised pipeline with new ablations over model depth and edge-aggregation direction.

Our study is guided by the following research questions:

- **RQ1** How effective is GNN-based C2A mapping compared to a Naive Bayes reference and a graph-free neural baseline?
- **RQ2** What does the dependency graph contribute beyond textual features under an identical training procedure, and which system properties govern where it helps?
- **RQ3** How sensitive is the approach to its design choices, and what does a full pipeline run cost as systems grow?

II. Related Work

Tzerpos and Holt [20] introduced the orphan adoption problem, in which source code entities that are not yet part of the documented architecture are incrementally assigned to modules. Christl et al. [10] built on this idea with the Human-Guided Clustering Method (HuGMe), a semi-automatic approach that recommends mappings through attraction functions and leaves the final decision to the architect. Bittencourt et al. [11] studied attraction functions derived from information retrieval, where an entity is assigned to the module whose textual content it most resembles. These approaches rely on two signals that persist in later work: lexical similarity between an entity and a module and dependency proximity to entities that are already mapped, but they combine them through fixed, manually specified rules.

Subsequent work replaced these rules with learned models. Olsson et al. [12] proposed a semi-supervised Naive Bayes mapper that combines lexical information from source code with dependency-derived information through Concrete Dependency Abstraction, and reported that 15–20% of the entities must be mapped beforehand to reach acceptable performance. Florean et al. [21] compared text classifiers for the same task and showed that both the classifier and the text representation affect mapping quality. InMap [13] instead ranks candidate mappings interactively, using natural language module descriptions together with source code, which removes the need for a pre-mapped training set. In all of these, dependencies enter the model only after being flattened into features attached to individual entities, so the relational structure of the system is consumed rather than represented.

Language models have recently been applied to the same task. Rukmono et al. [22] use chain-of-thought prompting with GPT-4 to deductively assign implementation units to a predefined reference architecture. Johansson et al. [23] evaluate language model embeddings alongside TF-IDF and GloVe as input to method-level classifiers, and in follow-up work attribute much of the variation across projects to lexical overlap and skew in the module size distribution rather than to the choice of model [24]. A parallel line applies language models to architecture recovery, where the module set is inferred rather than given [16, 17]. Pretrained models capture semantic cues that engineered text features miss, but their input remains predominantly textual, so typed dependencies stay implicit, and each entity is reasoned about largely in isolation. Prompting also scales poorly, since context limits and per-entity inference cost grow with the number of files.

Graph representation learning uses dependencies as structure rather than as features. NEGAR [25] showed that random-walk embeddings over the dependency graph, followed by clustering, recover meaningful architectural structure. GAER [14] extended this to heterogeneous graphs, comparing GCN and GAT encoders trained within a graph autoencoder and modeling each dependency type as a separate relation, and [15] reports related results using contrastive objectives. Both are unsupervised recovery methods, so the module set is discovered by clustering rather than fixed in advance as it is in mapping. Our earlier work [19] took this step and evaluated GCN-based models on homogeneous and dependency-type-aware graphs, showing that a small labeled set can be expanded through self-training. That study was limited to Java systems of moderate size, used a single graph-based encoder family, and did not isolate how much of the observed performance came from the dependency graph as opposed to the node features.

The present work addresses these limitations. We evaluate GNN encoders together with a graph-free neural baseline that receives identical features and an identical self-training loop, so any difference between them is attributable to the dependency graph rather than to representation or optimization. We extend the benchmark from Java to C/C++ and to an industrial-scale system, and report computational cost alongside mapping accuracy. Like the graph-based work discussed above, we operate at file-level granularity with a single node type, which

[1]Code and data are available at https://github.com/sse-lnu/c2a-graph.

keeps the comparison free of confounding from additional artifact types and leaves open what finer-grained entities contribute.

III. Methodology

This section presents our approach to C2A mapping as semi-supervised node classification over a directed multi-relational dependency graph. We first formulate the mapping task, then describe the graph and feature construction, introduce the encoders compared in this study, and finally the self-training procedure used to grow an initial labeling into a full mapping. Figure 1 summarizes the overall pipeline.

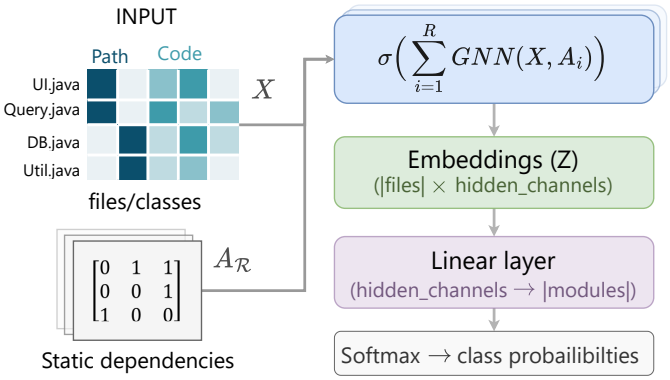


Fig. 1: Overview of the heterogeneous GNN encoder. Node features X (file-path indicator + Word2Vec identifiers) and relation-specific adjacency matrices $A_{\mathcal{R}}$ are passed through a relation-specific encoder (GCN or GAT), whose outputs are summed and passed through a non-linear activation function (σ) to produce embeddings $Z \in \mathbb{R}^{|\mathcal{F}| \times d_h}$. A linear layer maps Z to per-module logits, and a final softmax produces the class probabilities used for classification and pseudo-labeling.

A. Problem Formulation

Given a software system with a set of files $\mathcal{F}$ and a fixed set of architectural modules $\mathcal{C}$, a C2A mapping assigns each file $i \in \mathcal{F}$ to a module $y_i \in \mathcal{C}$. A small subset $L \subset \mathcal{F}$ is labeled with its ground-truth module, the remaining files $U = \mathcal{F} \setminus L$ are unlabeled, and the task is to recover the assignment for every file in U. The setting is transductive: the features and the dependency structure of every file are available during training, and only the labels are scarce.

Two kinds of evidence are available for each file. The first is intrinsic and consists of what the file itself declares (identifiers) and where it sits in the source tree (directory structure), and it is available for every file regardless of how the file is connected to the rest of the system. The second is relational: the modules assigned to its neighbors in the dependency graph, available only where the file has neighbors and only once those neighbors have been assigned. We frame the task as representation learning so the two sources can be treated separately: every encoder maps a file i to an embedding z_i, and a shared classification head maps z_i to a module, so the encoders differ in what evidence enters the encoder to generate z_i and not in how it is used.

B. Graph Construction

We represent a software system as a heterogeneous graph $G = (X, A_{\mathcal{R}})$ over its files, following the construction introduced for unsupervised recovery in GAER [14]. Code identifiers and directory paths are file-level properties and are encoded in the feature matrix X. Static dependencies are file-to-file relations, each belonging to a dependency type r drawn from the set of relation types $\mathcal{R}$, and encoded as edges in the multi-relational adjacency $A_{\mathcal{R}} = \{A^{(r)}\}_{r \in \mathcal{R}}$.

1) Node Features : The intrinsic evidence for a file comes from two conventions developers follow when they write and organize code. Identifiers name the concepts a file works with, so class, method, and variable names carry the vocabulary of the domain the file serves and, with it, an indication of the responsibility it implements [26]. Directory paths record how developers chose to group files, and that grouping frequently tracks the subsystem decomposition the architecture describes [9]. The two are complementary where structure alone is ambiguous, since a widely used utility file may depend on code throughout the system while its name and its location still indicate what it is for. Both signals are available from source text alone, without language-specific resolution, which is what allows the same feature construction to serve the Java and the C/C++ systems in our benchmark.

We normalize code identifiers by splitting on *camelCase*, *PascalCase*, and *snake_case* boundaries and removing low-information tokens (stopwords, very short tokens, tokens appearing in over 90% of files), then train a 128-dimensional skip-gram Word2Vec embedding over the remaining tokens for 20 epochs with a context window of 5. The textual representation T_i of a file i is the ℓ_2-normalized average of its token embeddings. File paths are tokenized separately and represented as a binary bag-of-words vector, which keeps each path component a distinct dimension so that a shared parent directory is visible to the model as a shared feature. For file i, the two are concatenated:

$$x_i = \left[\text{Binary(FilePath)}_i \,\middle\|\, \text{Word2Vec}(T_i)\right]. \tag{1}$$

Stacking x_i over all files gives the feature matrix X. Every encoder in this study receives the same X, so the intrinsic evidence available to each of them is identical by construction, and any difference between them originates in the relational evidence.

2) Static Dependencies (Edges) : Files that implement related functionality tend to depend on one another directly or through short dependency paths, which is what makes the dependency graph informative for architecture recovery [27]. Dependency types are not interchangeable in this respect, and call edges have been argued to indicate tighter coupling than type-usage edges [25, 28]. Collapsing them into one adjacency matrix forces the encoder to weight all of them alike, so we keep one adjacency matrix per type and let the encoder learn a separate transformation for each. Section V-C evaluates this

relation-typed design against a homogeneous graph for both GNN encoders.

We extract dependencies with the multilingual static-analysis tool *Depends* [29], which resolves the typed relations listed in Table I between code entities across our Java and C/C++ systems. Because the mapping task operates at file granularity, member-level dependencies such as a call between two methods are aggregated into edges between their enclosing files, and repeated dependencies of the same type between the same ordered pair collapse into a single edge, so $A^{(r)}$ records whether a relation holds and not how often it occurs.

TABLE I: Static dependencies used in the software graph.

Dependency	Description
`Annotation`	Annotation relation to an annotated entity.
`Call`	Function or method invocation.
`Cast`	Conversion of an entity to another type.
`Create`	Creation of an instance or program entity.
`Extend`	Inheritance or parent–child type relation.
`Implement`	A class implements an interface or abstraction.
`Import`	A file imports other classes, methods, or variables.
`Parameter`	Use of a type as a method or function parameter.
`Return`	Use of a type as a return type.
`Throw`	A type is used as an exception.
`Use`	Use or assignment of variables.

Annotation and Throw relations are not present in the analyzed C/C++ systems.

A static dependency is asymmetric, and $A^{(r)}$ preserves that direction: an edge points from the file where it originates to the file it refers to. For file i under relation r, we distinguish its in-neighborhood, the files that depend on it, from its out-neighborhood, the files it depends on. A file with a large in-neighborhood and a small out-neighborhood provides functionality, the profile of an interface or utility, and its clients may be spread across many modules. A file with the opposite profile consumes functionality, like orchestration or entry-point code, and what it draws on says more about what it coordinates than about which module it belongs to. Which of the two an encoder aggregates over, or whether it discards the direction altogether and treats the edge as undirected, is therefore a modeling choice rather than a fixed property of the graph; Section III-C defines it as a parameter of message passing, and Section V-C tests its impact on mapping.

C. Representation Learning

We compare three encoders that operate on the same node features X. The graph-based models, GCN and GAT, use the relation-specific adjacency $A_{\mathcal{R}}$ to aggregate information from neighboring files, while the non-graph baseline uses X alone. For each file i, all models take $h_i^{(0)} = x_i$ as input, stack K layers to produce embeddings $z_i = h_i^{(K)}$, and feed z_i into a shared linear classification head followed by softmax to obtain module probabilities. The encoders differ only in the neighborhood they aggregate over and in how they weight neighboring nodes.

1) GNN Encoders: Both GNN encoders update each file's representation by aggregating messages from its neighbors under each dependency type r, a mechanism known as message passing. The neighbors of file i under relation r form the set $\mathcal{N}_i^{r,\pi}$, where the propagation scheme π determines which side of the directed edge is aggregated over: forward, reverse, or undirected, as introduced in Section III-B2. A separate relation-specific kernel W^r transforms each neighbor's message, so a call edge and an inheritance edge contribute through different learned transformations rather than being folded into a single adjacency. Each additional layer widens this neighborhood by one hop, so a K-layer encoder lets information reach a file from up to K dependency steps away. We add self-loops before aggregation so a file's own features are included in its update, which matters when its neighborhood is empty. Forward propagation is the default in Equations 2 and 3.

GCN: For each file i, a GCN updates its representation at layer $k+1$ by weighing each neighbor's message by the two degrees of the two endpoints under that relation:

$$h_i^{(k+1)} = \mathrm{ReLU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \frac{1}{\sqrt{d_i^r d_j^r}} W^r h_j^{(k)} \Big), \qquad (2)$$

where d_i^r and d_j^r are the degrees of files i and j under relation r, and ReLU is the non-linear activation function. The degree normalization keeps high-degree files, such as widely used base types, from dominating the aggregation.

GAT: Compared with GCN, GAT [30] replaces fixed degree-based normalization with learned attention weights. Each neighbor's features are first projected via W^r, then scored and normalized with a softmax over $\mathcal{N}_i^{r,\pi}$ to obtain the attention coefficient α_{ij}^r:

$$h_i^{(k+1)} = \mathrm{ELU}\Big(\sum_{r\in\mathcal{R}} \sum_{j\in\mathcal{N}_i^{r,\pi}} \alpha_{ij}^r W^r h_j^{(k)} \Big). \qquad (3)$$

Unlike GCN, whose weights depend only on node degrees, GAT assigns weights based on neighbor features, which makes it more flexible but also adds parameters. This difference also affects the choice of propagation scheme π: in GCN, changing π changes only the neighbors being aggregated, while in GAT it changes both the neighbors and their learned weights.

2) Multi-layer Perceptron (MLP): To isolate the contribution of the dependency graph, we replace the GNN encoder with an MLP operating on x_i alone, with no neighbor aggregation. For each file i, the layer update rule is defined as:

$$h_i^{(k+1)} = \mathrm{ReLU}\big(\mathrm{BatchNorm}(W^{(k)} h_i^{(k)} + b^{(k)})\big). \qquad (4)$$

The GNNs build a file's representation from its neighbors, one or two hops away, depending on how many layers are stacked. The MLP builds it from the file alone, with no neighbors at all. Any performance gap between them is therefore attributable to the dependency graph itself, and not to differences in features or optimization, since all encoders are trained within the same iterative learning framework described next.

D. Iterative Learning Framework

All three encoders are trained with the same iterative procedure. They start from the same fixed set of labeled files L and grow it into a full mapping, with the remaining files forming the unlabeled set U. After initial training on L, the model predicts a module distribution for every file, and the confidence of a prediction is the maximum softmax probability over the module classes.

In each self-training round, unlabeled files whose confidence exceeds a threshold τ are assigned pseudo-labels and added to the mapped set $\mathcal{M}$. The model is then retrained on the original labeled files together with the accepted pseudo-labeled files. Existing pseudo-labels are revisited in the same round: if the model predicts a different module for a pseudo-labeled file with confidence above τ, its pseudo-label is updated. The procedure stops when no new confident pseudo-labels are found, when fewer than five files were added in the previous round, or when the maximum number of rounds is reached. Algorithm 1 summarizes the procedure, which is identical across the three encoders because the input representation is the only part that changes.

Algorithm 1: Iterative Mapping Algorithm.

Input: Input representation $\mathcal{I}$ (dependency graph G with node features X for GCN and GAT, node features X for MLP), set of files $\mathcal{F}$, initial labeled files L, confidence threshold τ, initial-training epochs E_w, self-training epochs E_s, max rounds R
Output: Trained classifier f, mapped files $\mathcal{M}$

```
M ← L;
U ← F \ L;
// Phase 1: train on initial labeled files only
Train classifier f on (I, M) for E_w epochs;
// Phase 2: self-training
added ← ∞;
for k ← 1 to R do
    if added < 5 then
        break;
    Score all files in U with f using I;
    foreach previously pseudo-labeled e ∈ M \ L do
        if confidence(e) ≥ τ and predicted label changed then
            Update pseudo-label of e;
    P ← {o ∈ U | confidence(o) ≥ τ};
    if P = ∅ then
        break;
    added ← |P|;
    M ← M ∪ P;
    U ← U \ P;
    Retrain f on (I, M) for E_s epochs;
return f, M;
```

IV. Experiments

A. Benchmark Systems

We rely on existing GT mappings rather than constructing new ones, since producing a reliable C2A mapping demands substantial architectural knowledge, manual effort, and often expert validation [9]. Table II lists the ten Java and C/C++ systems used in our evaluation. We also report file-level dependency homophily $\mathcal{H}$, the fraction of static dependency edges whose source and target files belong to the same ground-truth module, ranging from 0 (every dependency crosses modules) to 1 (every dependency stays within one). Each ordered pair of files with at least one static dependency between them contributes a single edge, regardless of how many individual dependencies or dependency types connect the pair.

TABLE II: Properties of the software systems. $\mathcal{C}$ denotes the number of modules in the GT architecture, edges are the static dependencies between files, and the average degree is computed as $\langle k \rangle = |E|/|\text{Files}|$.

	System	Version	LoC	$\mathcal{C}$	|Files|	$\lvert E\rvert$	$\mathcal{H}$	$\langle k \rangle$
Java	Ant	r584500	36k	15	815	12 028	0.37	14.8
	A.UML	r13713	62k	14	766	9 491	0.54	12.4
	C.Img	v1.0a2	18k	21	329	4 561	0.61	13.9
	JabRef	v3.7	59k	6	1 180	17 557	0.58	14.9
	Lucene	r1075001	36k	7	1 054	18 595	0.68	17.6
	T.Mates	v5.11	55k	15	778	17 818	0.33	22.9
C/C++	Bash	4.2	115k	14	292	1 222	0.60	4.2
	HDF	0e196f	153k	9	153	1 220	0.35	8.0
	HDC	46ff87	26k	11	207	1 117	0.63	5.4
	Chrome	svn-171054	11M	69	18 343	284 435	0.58	15.5

The Java systems are the same benchmark we used in our earlier work [19], assembled by Olsson et al. [12] from systems previously studied at SAEroCon and by Brunet et al. [31] and Lenhard et al. [32], with mappings distributed through the s4rdm3x GitHub repository[2]. For Bash, we use the C2A mapping curated by Garcia et al. [9], produced by deriving an initial decomposition from documentation and source code and then validating it with system experts. The same line of work provides the GT mapping for Chrome, established through roughly two years of collaboration with the system's developers [33]. The GT mappings for HDF and HDC were developed in collaboration with industry partners and are included in the SARIF benchmark studied by Zhang et al. [26], distributed through the SARIF replication package[3].

B. Experimental Configuration

GCN, GAT, and the MLP all use two layers by default, and the effect of using a single message-passing layer is examined separately in Section V-C. GCN and GAT use 256 hidden channels, and the MLP uses 256 hidden units. GCN and GAT apply a dropout rate of 0.01. GAT uses four attention heads. All three models are trained with the Adam optimizer at a learning rate of 0.001.

Training follows the same iterative learning procedure for every model: a warm-up phase on the initial labeled set of $E_w = 100$ epochs, followed by up to $R = 4$ rounds of self-training, each for $E_S = 30$ epochs. The initial labeled set is 5% of the entities in a system, bounded between 30 and 200 files. The minimum of 30 keeps results comparable across the smaller systems, where 5% alone would leave too few labeled entities to train on, while the maximum of 200 caps the initial set for large systems such as Chrome, where manually labeling 5% of the files would be impractical in a realistic setting. After

[2]https://github.com/h0bb3/s4rdm3x

[3]https://github.com/anonymous2f4a9d/SARIF_FSE23

sampling, we check that no module is left unrepresented in the initial labeled set. During self-training, only predictions with confidence above $\tau = 0.95$ are added as pseudo-labels. These settings are held fixed across all models. All experiments were run on a single HP ZBook Power G10 workstation with a 13th-generation Intel Core CPU and 32 GB of RAM.

C. Baseline: Naive Bayes

We compare against *NBAttract* [12] (NBA), a multinomial Naive Bayes classifier originally proposed for semi-supervised code-to-architecture mapping. NBA represents each entity using text features extracted from code identifiers and package names, combined with module-level Concrete Dependency Abstraction (CDA) sentences, document terms that encode cross-module structural relations, e.g., *A-Inherits-B* when an entity in module A inherits from one in module B. Olsson et al. report that NBA outperforms *CountAttract* [10] and the information retrieval methods *IRAttract* and *LSIAttract* [11] on the same benchmark [12], so we treat NBA rather than those earlier methods as the reference point for prior work. NBA was originally proposed and evaluated on Java systems only; we extend it to the C/C++ systems in our benchmark and reimplement the algorithm in Python, following the original iterative procedure, including its 0.90 confidence threshold for adding pseudo-labels and the same initial labeled set used by the other methods. Self-training stops either when no remaining entity can be mapped with sufficient confidence or when all entities have been mapped. Code for our NBA reimplementation and C/C++ extension is available in the GitHub repository 1.

D. Performance Evaluation

Module distributions in the benchmark systems are highly imbalanced, with several modules containing only one or two entities, so we evaluate mapping performance using precision, recall, and F1, computed with macro-averaging so that performance on small modules is not masked by larger ones. Precision captures how many of the entities assigned to a module actually belong to it, while recall captures how many of the entities that truly belong to a module were correctly assigned, and F1 is the harmonic mean of the two, penalizing cases where precision and recall are imbalanced more heavily than an arithmetic mean would. For F1, we report both macro- and micro-averaged scores: macro-F1 follows the same per-module averaging as precision and recall, while micro-F1 pools predictions across all entities and reflects overall system-level accuracy.

All scores are computed on the full test set, which comprises every entity outside the initial labeled set and covers pseudo-labeled and still-unmapped entities alike, so that a model leaving more entities unmapped cannot appear stronger simply by being scored on the easiest cases.

V. Results

We report results from 200 independent runs per model and system, with a randomly selected seed set used to initialize the training data for each run. All metrics are evaluated on the full test set, including unmapped entities, and summarized as the median across the 200 runs, with run-level variability shown as boxplots. The section addresses the three research questions in order: how effective GNN-based mapping is compared to a Naive Bayes reference and a graph-free neural baseline (RQ1); what the dependency graph contributes beyond node features under an identical training procedure, and which system properties govern where it helps (RQ2); and how sensitive the approach is to its design choices and what a full pipeline run costs as systems grow (RQ3).

A. RQ1: Overall Effectiveness of C2A Mapping

To answer RQ1, we compare NBA_{CDA}, MLP, GCN, and GAT under the same low-label setting and self-training loop over 200 independent runs per system. Table III reports the median macro and micro F1 on ten Java and C/C++ benchmark systems. NBA_{CDA} is included only as a reference from prior work, since all learned models substantially outperform it. The main comparison is therefore between MLP, GCN, and GAT, with MLP isolating the contribution of the node features without exploiting the dependency graph.

At a confidence threshold of 0.95, MLP maps a median of 28% of test files per system, compared with 96% for both GCN and GAT. Since evaluating only mapped files would favor lower-coverage methods, all results in Table III are reported on the full test set.

GAT is the best-performing method on micro F1, achieving the best or tied-best result on eight of the ten systems, including all four C/C++ systems. Macro F1 splits the benchmark: GAT leads on four Java systems, MLP on five, with one tie, and on the C/C++ side MLP leads on Bash, HDF, and HDC. Those three are the smallest and least connected systems in the benchmark, with average node degree of 4.2, 8.0, and 5.4, so message passing has little to aggregate over (Section V-B), and design choices such as aggregation depth matter most there (Section V-C). The near-equal macro averages therefore mask two groups with opposite outcomes rather than a uniformly small gap: GAT's largest margins fall on Chrome and JabRef, MLP's on Lucene and HDF. Chrome is the only large-scale system in the benchmark and shows the widest gap in either direction on both metrics, 16 and 14 macro F1 points for GAT and GCN over MLP, so the advantage of the graph grows rather than shrinks with system size. GCN sits between the two almost everywhere, rarely best but consistently far ahead of NBA_{CDA}.

The C/C++ results come from the same pipeline and the same typed dependency relations, with no language-specific tuning, and they follow the Java pattern for the same reasons: MLP's advantage tracks low connectivity rather than language, and the gaps compress only on the small, sparse systems, where limited connectivity caps both what message passing can add and how far a graph-free model can fall behind. The approach therefore generalizes across languages, with the same properties deciding outcomes in both. Section V-B

TABLE III: Median F1 (×100, 200 runs). MA = macro F1, MI = micro F1. The best score for each (system, metric) pair is highlighted.

	Java												**C / C++**							
	Ant		A.UML		C.Img		JabRef		Lucene		T.Mates		Bash		HDF		HDC		Chrome	
Method	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI	MA	MI
NBA_{CDA}	16	44	31	62	20	53	40	81	53	75	23	60	21	44	45	55	45	72	–	–
MLP	79	84	86	93	89	91	65	80	95	97	90	94	72	87	93	94	74	87	65	66
GCN	82	85	82	93	91	94	76	94	85	94	77	91	69	88	79	83	68	89	79	89
GAT	85	88	86	95	92	96	79	94	88	95	88	96	69	88	83	86	72	91	81	90

examines what separates the two GNNs from the MLP system by system.

Statistical significance of observed differences: To confirm that the differences in Table III reflect genuine performance gaps, we compare the three learned methods pairwise using the Mann–Whitney U test [34] on the paired macro F1 distributions across 200 independently sampled runs. To quantify magnitude we report Cliff's δ [35], interpreted by the standard convention: negligible ($\delta < 0.147$), small ($0.147 \leq \delta < 0.33$), medium ($0.33 \leq \delta < 0.474$), and large ($\delta \geq 0.474$). Table IV reports the results. GAT outperforms GCN on every system except Bash, where the difference is not statistically significant, with an average gain of $+3$ macro F1 points at a small effect size. The pattern against MLP is different in kind rather than in degree: the near-zero macro average of $+1$ point masks a bimodal outcome, with GAT outperforming significantly and with large effect sizes on some systems and under-performing significantly and with comparable effect sizes on others, so the two methods are rarely close on any single system despite averaging close overall. The GCN comparison shows the same split, with MLP's aggregate lead of -2 points driven by the same systems alongside a decisive GCN advantage on Chrome at $\delta = 1.0$. When measured by micro F1, GAT's average advantage over MLP widens to $+4$ points, confirming that the macro-level losses reflect difficulty on minority modules in specific systems rather than a general accuracy deficit.

TABLE IV: Pairwise comparison of methods across 200 runs per (system, metric) cell. Each entry reports the median difference between the two methods (Δ), marked with † when the difference is not statistically significant ($p \geq 0.05$), and Cliff's delta (δ) with its effect-size class in parentheses (L: large, M: medium, S: small, N: negligible). Positive values favor the first method; negative values favor the second. Cells in which the second method is significantly better than the first are highlighted.

	GAT − GCN		GCN − MLP		GAT − MLP	
System	Δ	δ	Δ	δ	Δ	δ
Ant	$+3$	0.3 (S)	$+3$	0.2 (S)	$+5$	0.5 (M)
A.UML	$+4$	0.4 (M)	-4	0.4 (M)	$0^{\dagger}$	0.0 (N)
C.Img	$+1$	0.3 (S)	$+2$	0.2 (S)	$+3$	0.5 (L)
JabRef	$+3$	0.2 (S)	$+11$	0.7 (L)	$+13$	0.8 (L)
Lucene	$+3$	0.2 (S)	-10	0.9 (L)	-7	0.8 (L)
T.Mates	$+12$	0.8 (L)	-13	0.9 (L)	-2	0.2 (S)
Bash	$0^{\dagger}$	0.0 (N)	-3	0.2 (S)	-3	0.2 (S)
HDF	$+4$	0.4 (M)	-13	0.9 (L)	-10	0.8 (L)
HDC	$+4$	0.2 (S)	-6	0.4 (M)	-3	0.2 (S)
Chrome	$+2$	0.4 (M)	$+14$	1.0 (L)	$+16$	1.0 (L)
Avg. gain	$+3$	0.3	-2	0.6	$+1$	0.5

How does the graph reshape the model's operating point? Figure 2 plots one point per run in the macro precision–recall plane. Precision captures the purity of a recovered module, the share of files assigned to it that truly belong. Recall captures its completeness, the share of its true members that were found. The two models sit on opposite sides of this tradeoff. MLP's recall exceeds its precision on eight of the ten systems, while GAT's precision exceeds its recall on nine. File names and folder paths often carry across module boundaries, which would let lexical features pull in neighbors along with members. In contrast, message passing draws a file into a module when its dependencies support it. Overall, GAT returns modules that can be trusted but must be completed, and MLP returns modules that are close to complete but must be checked.

How stable is mapping across different initial training draws? Figure 3 shows the micro F1 distributions across 200 runs. Spread is narrow for all learned methods, with interquartile ranges under 6 points on seven of the ten systems. Macro F1 spread is wider on the same runs, because minority modules are the ones most affected by which files happen to be drawn. Where variability appears, it appears across all three methods together, indicating that the driver is the system rather than the model. HDC shows the widest spread: macro F1 interquartile ranges reach 14 to 15 points for GAT and GCN against 7 points on micro F1, so instability concentrates on rare minority-class predictions that macro weights equally. MLP produces the tightest macro distribution on several systems (Ant, A.UML, HDF) but not uniformly, so GAT's advantage over MLP lies in the median rather than in reduced variance.

Variability also tracks the absolute number of labeled files available at initialization and interacts with graph sparsity, not with system size or language. The three sparsest systems are also the three smallest (HDC, HDF, Bash, all under 300 files), and they show the widest spread: their 30-file floor spreads only one or two labeled files across each module, and with few dependencies per file the models have little structural signal to recover from an unlucky draw, so run-to-run performance depends heavily on which specific files are selected. C.Img provides contrast, since it is also small (329 files), dense and cohesive, and remains stable. Once the initial training set reaches roughly 50 files, interquartile

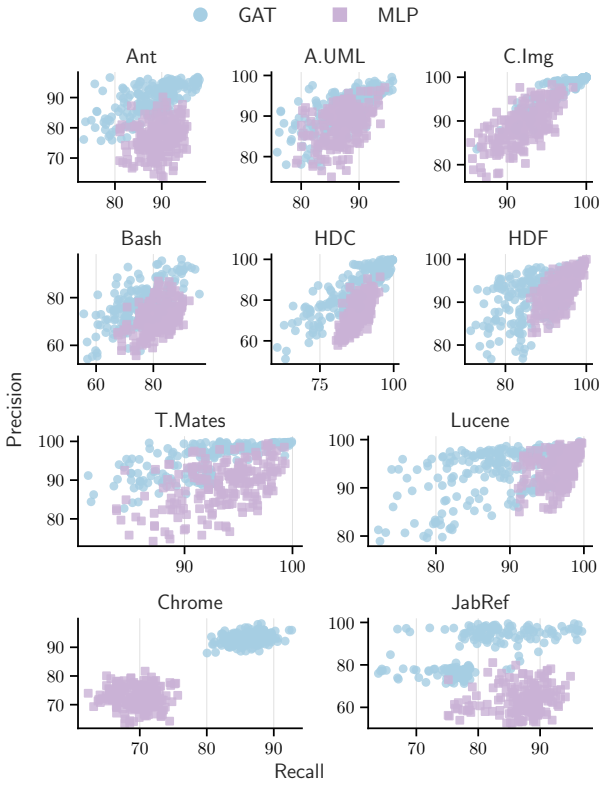


Fig. 2: Per-run macro precision and recall for GAT and MLP across systems, one point per run. GCN is omitted for clarity and follows GAT's pattern at slightly lower precision and recall.

ranges tighten sharply and stay tight regardless of scale or complexity: Chrome, the largest and most fine-grained system (69 modules) in the benchmark, has the tightest macro F1 spread of all (GAT interquartile range 3.3). The residual macro F1 variability tracks module-size skew rather than the size of the initial training set, since systems that mix very large and very small modules (e.g., JabRef) exhibit a wider macro F1 spread even with adequate supervision.

Answer to RQ1 All learned models beat NBA_{CDA} by a wide margin. GAT reaches the highest average macro F1 (82.2), ahead of MLP (80.8) and GCN (78.7), and is best or tied for best on micro F1 on eight of the ten systems. GCN sits between the two on micro F1, but behind both on macro F1, so degree normalization alone does not reliably beat the graph-free baseline; the gain from the graph comes with attention. GAT and MLP fall on opposite sides of the precision–recall tradeoff, GAT favoring precision and MLP recall. Run-to-run variation drops sharply once the initial training set reaches about 50 files, regardless of system size or language, and the same ordering holds on the four C/C++ systems without language-specific adjustment.

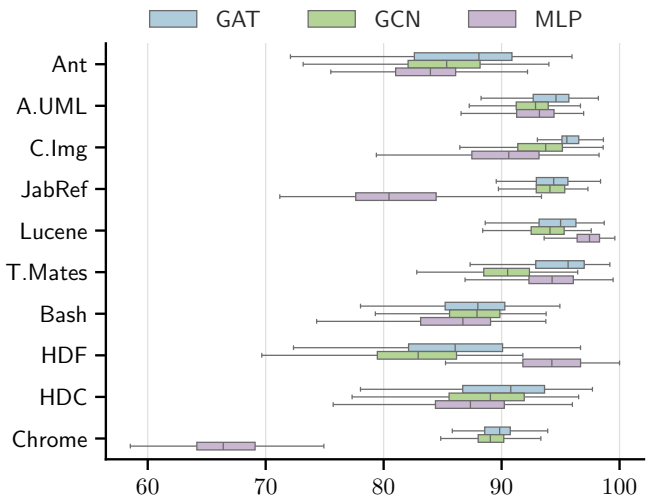


Fig. 3: Distribution of micro F1 scores across 200 runs for each model and system. Variability across runs stems mainly from the random initial labeled-set draw, with model weight initialization contributing a smaller share.

B. RQ2: Contribution of the Dependency Graph

RQ1 establishes that neither GAT nor MLP consistently outperformed the other across all systems. RQ2 asks what the dependency graph actually contributes beyond lexical features, in two parts. First, we identify which system properties decide where message passing helps and where it does not. Second, we test whether the graph makes file-level assignments more repeatable, that is, whether the same file lands in the same module across different initial training sets, a question median F1 cannot answer because it collapses the whole system into one number. All models use identical lexical features and training loop, so any systematic difference is attributable to the graph, message passing, and attention.

Why do MLP and GAT outperform each other on different systems? Message passing helps most when a file has a neighborhood to aggregate and that neighborhood is cohesive, meaning connected files mostly belong to the same module. Standard GCN and GAT assume that connected nodes tend to share a label, so aggregation reinforces the local signal when modules are cohesive; under heterophily, the same aggregation pulls a representation toward files in other modules [36]. Neither property decides the outcome on its own. Bash and HDC are as cohesive as any system in the benchmark, yet MLP outperforms both GNNs on macro F1 on each, and they also have the lowest average node degree, so a file with few dependencies offers too little neighborhood for aggregation to exploit whatever cohesion is present. Ant makes the converse point, with low cohesion but a well-connected graph, where GAT still gains five macro F1 points over MLP. Therefore, connectivity determines whether the graph can contribute at all, and cohesion modulates how much.

Lucene and T.Mates have well-connected graphs, yet MLP

outperforms both GNNs, so sparsity is not the only route to an MLP advantage. In these systems, the lexical features align closely with module boundaries. Elsewhere, the deciding property is how evenly files are spread across modules. MLP performs best when modules are close to equal in size and degrades for the imbalanced module distributions. Lucene and HDF have the most evenly sized modules and are where MLP's margin is largest, while Chrome and JabRef are the most skewed and where it falls furthest behind, with fifty-two of Chrome's sixty-nine modules holding under one percent of the files each and a single JabRef module holding sixty percent of its files. Lexical features have to separate small modules from few examples, while message passing supplies evidence that does not depend on how many files a module contains. GAT's margin also widens as systems scale, and Chrome combines the largest codebase in the benchmark with the most modules and the most uneven distribution among them, so scale favors the graph rather than straining it.

Does message passing make individual-file decisions more repeatable? For every file, we compute its misclassification rate: the fraction of the 200 runs in which it was assigned the wrong module, counting only the runs where it was held out for testing. We call a file never-wrong when this rate is zero, persistently wrong when it is at least 0.8, and occasionally wrong otherwise. The files are held out for testing in a median of 198 of the 200 runs, so differences in misclassification rate are not an artifact of unequal exposure. Figure 4 shows the cumulative distribution of misclassification rates for each model and system.

The curves separate differently across systems rather than uniformly along a single GNN-versus-MLP axis. On most systems, message passing raises the never-wrong share several-fold over MLP, most dramatically on Chrome where GAT and GCN place 31% and 27% of files in the never-wrong bucket, files correctly classified in every one of the 200 runs, against 0.09% for MLP. The median file on Chrome is misclassified in 1.5% of runs under GAT and 31% under MLP. Message passing does not make every file a little more accurate; it takes files whose local dependency neighborhood is homophilic, meaning most neighbors share the file's module, and makes them reliably correct across different training draws.

Lucene and HDF run the other way, and both are systems on which MLP already outperformed the GNNs on F1 in RQ1: Lucene because folder paths resolve modules on their own, and HDF because the graph is too sparse to aggregate over. The two GNNs also diverge in ways that F1 does not predict. On JabRef, GCN places 69% of files in the never-wrong bucket against GAT's 53%, even though GAT has the higher median macro F1, so attention improves which module a file lands in more than how consistently the file lands there.

The persistently wrong tail behaves differently. It stays below 3% across all models on all systems, and the differences between models are small on nine of the ten systems. HDF is the exception: MLP has no persistent tail there, whereas GCN and GAT reach 4.6% and 2.6%, respectively. Message passing therefore stabilizes files whose neighborhoods are homophilic,

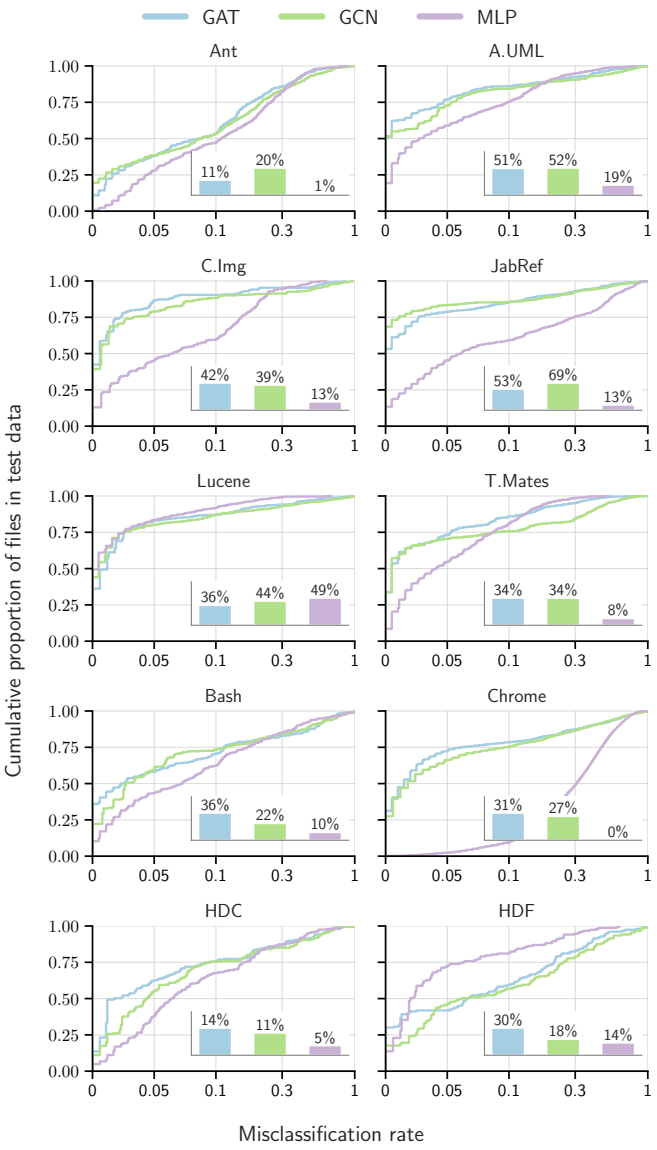


Fig. 4: Empirical cumulative distribution of per-file misclassification rates across 200 runs, one panel per system, for GAT, GCN, and MLP. The y-axis is the cumulative proportion of test-set files, pooled across runs, whose misclassification rate is at or below the value on the x-axis, so a curve that rises earlier and higher indicates more reliable per-file decisions. The x-axis is compressed above 0.1 so the near-zero region, where the models differ most, is easier to read. The inset bar chart in each panel shows the never-wrong percentage (misclassification rate = 0%) for each model, rounded to the nearest percent.

and on the sparse graphs (Bash, HDC, HDF) it misclassifies files whose neighborhoods are heterophilic or too thin to carry a useful signal. The remainder of this subsection examines what those files have in common.

What do the graph models get persistently wrong? Persistently wrong files (misclassification rate ≥ 0.8) are rare, at

most a few percent of any test set, but they are not a single phenomenon. We analyzed the dependency neighborhood of 414 such files and found two dominant patterns that account for 91% of them. The patterns differ in how many neighbors a file has, not in whether those neighbors point to the right module. In 53% of the cases, the file has many neighbors, and most of them belong to a module other than its own, so aggregation carries the wrong module into its representation. In 38% of the cases, the file has almost no neighbors, so aggregation has no evidence to work with, and the model falls back on node features alone. The remaining 9% are files whose neighborhoods are split between modules.

Table V illustrates the first pattern with nine cases from Chrome, Lucene, and JabRef. Each file has far more incoming than outgoing edges, and its neighborhood is dominated by the module of its callers rather than its own. Incoming edges outnumber outgoing edges by an order of magnitude or more, reaching several hundred to one in the extreme cases. Across the nine cases, the file's true module accounts for at most 22% of its neighborhood, while the predicted module accounts for at least 68%. This holds whether the folder path indicates the true module (Chrome, Lucene) or the path is uninformative (JabRef). The pattern holds across all 219 files in this category: incoming edges outnumber outgoing edges in 94% of them, with a typical file having about 87 incoming edges and 7 outgoing edges. These are interface and utility files, defined by what they provide to the rest of the system rather than by what they use. Message passing then shapes their representation from the callers that depend on them, and if those callers mostly belong to another module, the file inherits that module.

Furthermore, the predicted module tends to be one of the larger ones in the system in these cases, since a large module supplies more callers than a small one and therefore dominates more neighborhoods. This does not reduce to a prior over module frequency: eight of the nine files in the Table are predicted to be a module other than the largest, and in each case, that module is the one that supplies the callers.

The second pattern is the reverse: files that are isolated or have a very sparse neighborhood in the dependency graph. Across all 156 files in this category, 47% have no incoming edges and 23% have no edges of any kind, with a typical file receiving 0 incoming and 4 outgoing edges. Message passing has no neighbors to aggregate, and the model falls back on node features that already proved insufficient to place these files correctly. MLP fails on the same files by the same measure: its median misclassification rate on this set is 0.82, comparable to GAT and GCN, so no model recovers these files reliably regardless of whether it uses the graph. This is the file-level counterpart of the finding above that MLP matches or beats both GNNs on the three sparsest systems: files with no dependencies to aggregate defeat both approaches, and the graph adds nothing a features-only classifier could not already fail at. The two patterns are limitations of different kinds. Files in the first are misclassified because the graph carries a strong signal that disagrees with the ground-truth module, which is a limitation of source-to-target aggregation on directed dependency graphs rather than a shortage of evidence. Undirected and reverse-edge variants do not fix this (Table VII), so the asymmetry is not simply removable, and whether it can be addressed by weighting a node's own features more heavily remains an open question. Files in the second are misclassified because the graph carries no signal that any aggregation scheme can repair. These are the extreme cases, but the same conditions shape the middle of the distributions (Figure 4): a file is hard whenever its features do not separate its module from others and its neighborhood either misleads aggregation or supplies nothing to aggregate.

Answer to RQ2 The graph contributes in two ways beyond lexical features. First, it decides where message passing helps at all: connectivity determines whether a file has enough neighborhood to aggregate, and cohesion determines whether that neighborhood supports its true module. Where both conditions hold, GAT and GCN are also less sensitive to imbalanced module distribution than MLP. Second, the graph makes file-level decisions noticeably more repeatable across different initial training sets. On nine of the ten systems, message passing places several times as many files in the never-wrong bucket as MLP does, and attention improves which module a file lands in more than how consistently it lands there. The persistently wrong tail stays small under every method and does not shrink when the graph is added, since these files sit in a neighborhood dominated by another module or have none at all, and their own features do not carry enough signal to correct for it.

C. RQ3: Design Sensitivity and Scalability

RQ1 and RQ2 establish where the approach works and what the graph contributes. RQ3 turns to the conditions under which it holds: whether modeling dependency types separately is necessary, how sensitive the approach is to the two design choices that govern message passing, the number of layers and the direction of aggregation, and what a full pipeline run costs as systems grow.

Does modeling dependency types separately matter? Our encoders keep one adjacency matrix per relation type and learn a separate transformation for each, rather than collapsing all dependencies into a single edge type. Figure 5 compares this multi-relational design against a uni-relational variant, in which all dependency types are merged into one undifferentiated adjacency matrix before message passing, for both GAT and GCN across the ten benchmark systems.

Multi-relational aggregation outperforms the uni-relational variant on every system for both encoders, with median macro F1 gains of 17 to 41 points for GAT and 12 to 31 points for GCN. This reinforces that every dependency type carries different structural signal about module membership: a call edge and an inheritance edge do not imply the same likelihood that two files share a module, so collapsing them into a single relation forces the encoder to transform both with the same weights. Keeping one adjacency matrix per relation type lets the encoder learn a separate transformation for each relation.

TABLE V: Dependency-neighborhood composition for persistently misclassified files in the path-overridden category (GAT). GT% is each module's share of files in the system. In/Out are the file's incoming and outgoing dependency edges (de-duplicated by source, target, and dependency type). True/Pred. share is the fraction of the file's neighborhood, pooled across both directions, in its true or predicted module. † marks the row where the predicted module is the system's largest.

System	File	True (GT%)	Predicted (GT%)	Rate	In	Out	True share	Pred. share
Chrome	v8/src/unbound-queue-inl.h	v8base (1.7%)	webcore_webkit (23.3%)†	0.98	896	2	0.003	0.991
	content/.../notification_source.h	content (7.3%)	browser (20.6%)	0.96	644	2	0.050	0.949
	ui/base/ui_base_types.h	ui (1.6%)	browser (20.6%)	1.00	38	3	0.073	0.927
	base/values.h	base (2.1%)	browser (20.6%)	1.00	2840	6	0.021	0.771
Lucene	store/Directory.java	store (6.2%)	index (19.9%)	0.87	280	17	0.155	0.758
	store/IndexOutput.java	store (6.2%)	index (19.9%)	0.87	194	1	0.200	0.718
	store/IndexInput.java	store (6.2%)	index (19.9%)	0.88	243	5	0.202	0.681
JabRef	JabRefExecutorService.java	logic (60.9%)	gui (29.2%)	0.97	48	2	0.040	0.940
	PreviewPreferences.java	preferences (0.9%)	gui (29.2%)	0.87	18	0	0.222	0.778

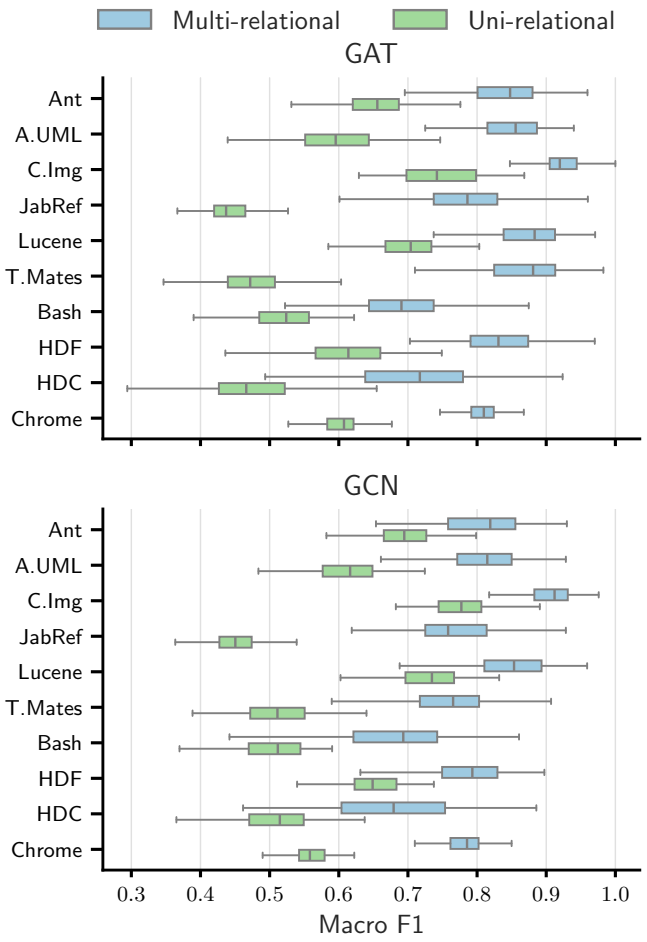


Fig. 5: Distribution of macro F1 across 200 runs for the multi-relational and uni-relational variants of GAT and GCN, one panel per model, across all ten systems.

This is why separating relation types, not the dependency graph alone, drives much of the improvement over the graph-free baseline.

How many message-passing layers does the graph support? Stacking more layers in a GNN does not improve accuracy the way added depth can in a feedforward network. Each layer widens the receptive field[4], and once that field spans a large part of the graph, repeated aggregation drives node embeddings toward a common value and they lose the discriminative power the classifier needs, an effect known as over-smoothing [37]. We therefore explore only one and two layers, and Table VI reports the comparison.

TABLE VI: Effect of aggregation depth on median macro F1 across 200 runs for the GNNs. Two layers are the default, and the two-layer columns reproduce the GCN and GAT medians in Table III. Best value per (system, model) pair is highlighted.

	2 layers		1 layer	
System	GCN	GAT	GCN	GAT
Ant	82	85	71	74
A.UML	82	86	79	81
C.Img	91	92	91	93
JabRef	76	79	62	67
Lucene	85	88	82	85
T.Mates	77	88	79	86
Bash	69	69	63	64
HDF	79	83	79	83
HDC	68	72	77	80
Chrome	79	81	77	78

Two layers perform better on six systems for GCN and seven for GAT, and the largest gains appear on JabRef (+14 GCN, +12 GAT) and Ant (+11 for both). Choosing the wrong depth costs little on most systems: C.Img and HDF differ by at most one point between the two depths, and T.Mates by two. HDC is the only system where the second layer clearly hurts, dropping 9 points for GCN and 8 for GAT. No property we report in Table II predicts this. HDC is small and sparse, which is consistent with a second hop that averages over a wider set without adding new structure, but Bash is smaller and sparser still and gains 6 points from the second layer. Two layers is therefore the right default, and depth is the one setting worth checking on the smallest and sparsest systems.

[4]The receptive field of a node is the set of nodes within L hops that can influence its representation through L layers of message passing.

Does edge direction matter? The heterogeneous encoder aggregates source-to-target by default, so a file's representation is shaped by the neighbors that depend on it rather than by the neighbors it depends on. Table VII compares this against an undirected variant, which symmetrizes every edge, and a reverse variant, which aggregates target-to-source.

TABLE VII: Effect of edge direction on macro F1 (median $\times 100$). Directed (D) is the source-to-target aggregation used throughout; Undirected (UD) symmetrizes every edge; Reverse (RD) aggregates target-to-source. Best value per (system, model) row is highlighted.

	GAT			GCN		
System	D	UD	RD	D	UD	RD
Ant	84.8	85.1	84.7	81.9	82.0	82.4
A.UML	85.6	83.6	82.9	81.5	81.9	82.1
C.Img	92.0	88.8	91.0	91.2	87.5	89.1
JabRef	78.6	71.9	72.0	75.8	77.2	73.1
Lucene	88.3	85.0	88.0	85.3	87.6	85.6
T.Mates	88.1	77.2	79.8	76.5	77.8	72.8
Bash	69.1	71.8	71.4	69.3	71.5	70.6
HDF	83.1	78.7	80.2	79.3	78.9	77.8
HDC	71.7	66.8	66.8	67.9	67.5	69.0
Chrome	81.0	75.9	76.5	78.5	77.5	77.5

The two encoders respond to direction very differently. For GAT, source-to-target aggregation, which aggregates over the files that depend on it (in-neighborhood), performs best on eight of the ten systems. The margins are large on several: T.Mates gains almost eleven points over the undirected variant, and JabRef and Chrome gain five to seven. Aggregating over the in-neighborhood means that a file's representation is shaped by its clients, and when those clients share a module, the attention mechanism can reinforce the correct assignment. The reverse and undirected variants mix in the out-neighborhood, where a file's own dependencies may reach into other modules and dilute that signal. Bash is the one system where undirected aggregation performs best for both encoders, with a margin under three points. GCN is largely direction-agnostic across all systems: no variant outperforms the others by more than two to three points anywhere, because degree normalization applies the same weight to every neighbor, so changing which neighbors are visible changes little about how they are combined. Changing direction does not fix the files that RQ2 found are persistently misclassified. Those errors come from the large gap between a file's incoming and outgoing dependencies, and flipping or symmetrizing the edges does not close that gap.

How does the approach scale? To assess scalability, we report the total time consumption of a full pipeline run for each method, averaged over five runs (Table VIII). A run covers node feature generation, graph construction for the GNNs, and the complete iterative training procedure; only the extraction of static dependencies is excluded, since it is performed once per system with Depends and reused across methods. MLP is fastest throughout, as it neither builds the dependency graph nor performs message passing. GCN adds message passing over typed edges, and GAT adds a learned attention weight per edge, so the ordering holds on every system. Chrome, the largest system in the benchmark at 18,343 files, completes a full GAT run in under two and a half minutes, while every other system finishes in under 40 seconds, so the approach is scalable to systems of industrial scale.

TABLE VIII: Total time consumption (in seconds) averaged over five runs.

	System	MLP	GCN	GAT
Java	Ant	14.99	21.15	25.48
	A.UML	21.73	25.85	32.06
	C.Img	2.83	7.44	12.86
	JabRef	2.20	9.07	10.93
	Lucene	15.78	19.82	23.48
	T.Mates	30.29	36.95	37.98
C / C++	Bash	0.93	3.77	7.27
	HDF	0.71	4.94	8.03
	HDC	0.99	3.78	5.90
	Chrome	41.66	127.98	141.38

Answer to RQ3 Different dependency types carry different architectural signals: multi-relational aggregation outperforms a single merged edge type on every system for both encoders, by a wider margin. On depth, two message-passing layers are the right default, except on small, sparse graphs where the second hop over-smooths across module boundaries. On direction, source-to-target aggregation is the better direction for GAT, performing best on eight of ten systems; GCN is largely insensitive. None of these choices resolves the persistently misclassified files from RQ2, motivating two directions for future work: weighting a file's own features against its neighborhood for interface files that inherit their callers' module, and testing whether these findings extend to dynamically typed languages, where dependency extraction is less precise. Finally, a full GAT run completes in under 40 seconds on medium-sized systems and under two and a half minutes on Chrome, so the obstacle to automated C2A mapping is the graph's usable signal, not computation cost.

VI. Threats to Validity

External Validity: The benchmark spans Java, C, and C++ systems for which a published module mapping exists. Availability of ground truth, rather than representativeness, drives this selection, so the systems do not evenly sample architectural styles, application domains, or team practices. Dynamically typed languages are absent, and in those languages a static extractor recovers a sparser graph, which would shift the balance between the graph-based models and the graph-free baseline in a direction our results cannot predict. Chrome is our only system at industrial scale, so what we report about scaling rests on one observation rather than a trend. None of the systems consist largely of AI-generated code, and whether such code produces dependency structures with comparable module cohesion is untested.

Internal Validity: The dependency graph comes from a static analyzer that resolves references syntactically. Reflection, function pointers, conditional compilation, and macro expansion produce interactions it cannot see, and unresolved symbols can yield edges that no execution realizes. Since the two GNN encoders draw their advantage from exactly these edges, extraction errors bias against them rather than against the baseline, which makes our reported graph contribution conservative rather than inflated. Matching the source revision used in earlier studies is another source of error, as several benchmarks predate their published mappings by years, and files generated at build time enter or leave the graph depending on how the system was configured.

Ground-truth mappings carry the judgment of whoever produced them. Some came from developers or industry partners with direct knowledge of the system, others were reconstructed by researchers from documentation and code inspection, and files that plausibly belong to two modules are resolved differently by different annotators. Because the initial labeled set is drawn from the same mapping used for evaluation, an annotation error can both mislead training and penalize a correct prediction.

Our unit of analysis is the file. This follows the granularity of the available ground truth and keeps graph construction tractable, but a file implementing more than one concern must still receive a single label, and no model in our comparison can express that ambiguity.

Comparison against NBA required a Python reimplementation extended to C/C++, since the original targets Java. We preserved the published procedure and supplied it with the same labeled files given to the other models, though we cannot fully exclude divergence from the original code.

Finally, we report uncorrected p-values across the thirty pairwise comparisons, so individual results near the threshold should be read alongside the effect size rather than on significance alone.

Construct Validity: Node features combine code identifiers with directory context, and in several systems the directory tree mirrors the intended decomposition. Some of the reported accuracy may therefore reflect naming and layout conventions rather than structure recovered from the code. All three models receive identical features, so the comparison between them holds, but absolute scores are optimistic wherever folders track modules closely.

Whether a file receives a pseudo-label depends on a fixed confidence threshold, which reflects how sharply a model concentrates its softmax output and not only how well it maps files. Reporting scores over the complete test set prevents a model that maps fewer files from benefiting from an easier evaluation set.

Timing covers feature generation, graph construction, and training on one hardware configuration. The absolute figures are specific to that setup, and only the relative ordering across models and systems transfers.

VII. Conclusion

We studied code-to-architecture mapping as semi-supervised node classification over a directed multi-relational dependency graph, comparing GAT and GCN against an MLP baseline that receives the same lexical features and passes through the same self-training loop, so that differences can be attributed to the dependency structure itself. Across ten systems in Java and C/C++ and 200 runs per system, all three learned models improve on the Naive Bayes reference by a wide margin, and GAT reaches the highest average macro F1, ahead of MLP and GCN, with the same ordering holding across both languages. The advantage of the graph grows rather than shrinks with scale: on Chrome, the largest system in the benchmark at 18,343 files, GAT and GCN outperform MLP by 16 and 14 macro F1 points.

Message passing also makes individual file-level decisions markedly more repeatable across different initial training sets, placing several times as many files in the never-wrong category as MLP does across most systems.

Modeling dependency types separately matters more than any other design choice tested: different edge types carry different structural signal, and letting the encoder learn a separate transformation per relation outperforms a single merged edge type on every system. Two layers and source-to-target aggregation are the right remaining defaults, and a full run completes in under 40 seconds on medium-sized systems and under two and a half minutes on Chrome, so scale is not what limits the approach.

What limits it is how much module signal the graph carries, and this points to several directions: making attention itself sensitive to relation type, since GAT currently learns its weights over node features alone and cannot yet favor one dependency type over another when they disagree; enriching the graph with finer-grained nodes for folder hierarchy and class members, closer to how architects describe modules; and isolating how much architectural signal the lexical features carry on their own. Together, these target the residual failure modes from RQ2: files whose neighborhoods contradict their true module and files with too little structural evidence to place.

References

[1] P. Clements, D. Garlan, R. Little, R. Nord, and J. Stafford, "Documenting software architectures: views and beyond," in *25th International Conference on Software Engineering, 2003. Proceedings.* IEEE, 2003, pp. 740–741.

[2] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012. [Online]. Available: https://philippe.kruchten.com/wp-content/uploads/2012/08/kruchten-120821-techdebt.pdf

[3] R. Ehsani, A. Aljohani, H. Do, and Y. Cai, "Faster code, deeper debt? A multivocal literature review on technical debt and its early signs in LLM-assisted software development," *ACM Transactions on Software Engineering and Methodology*, 2025.

[4] W. Wu, H. Hu, Z. Fan, Y. Qiao, Y. Huang, Y. Li, Z. Zheng, and M. R. Lyu, "An empirical study of code clones from commercial AI code generators," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 2874–2896, 2025.

[5] L. De Silva and D. Balasubramaniam, "Controlling software architecture erosion: A survey," *Journal of Systems and Software*, vol. 85, no. 1, pp. 132–151, 2012.

[6] G. C. Murphy, D. Notkin, and K. Sullivan, "Software reflexion models: Bridging the gap between source and high-level models," in *3rd ACM SIGSOFT Symposium on Foundations of Software Engineering*, 1995, pp. 18–28.

[7] M. V. Moreira and J. C. Basilio, "Bridging the gap between design and implementation of discrete-event controllers," *IEEE Transactions on Automation Science and Engineering*, vol. 11, no. 1, pp. 48–65, 2013.

[8] N. Ali, S. Baker, R. O'Crowley, S. Herold, and J. Buckley, "Architecture consistency: State of the practice, challenges and requirements," *Empirical Software Engineering*, vol. 23, pp. 224–258, 2018.

[9] J. Garcia, I. Krka, C. Mattmann, and N. Medvidovic, "Obtaining ground-truth software architectures," in *35th International Conference on Software Engineering (ICSE)*. IEEE, 2013, pp. 901–910.

[10] A. Christl, R. Koschke, and M.-A. Storey, "Automated clustering to support the reflexion method," *Information and Software Technology*, vol. 49, no. 3, pp. 255–274, 2007.

[11] R. A. Bittencourt, G. J. d. Santos, D. D. S. Guerrero, and G. C. Murphy, "Improving automated mapping in reflexion models using information retrieval techniques," in *17th Working Conference on Reverse Engineering*, 2010, pp. 163–172.

[12] T. Olsson, M. Ericsson, and A. Wingkvist, "To automatically map source code entities to architectural modules with naive bayes," *Journal of Systems and Software*, vol. 183, p. 111095, 2022.

[13] Z. T. Sinkala and S. Herold, "Inmap: Automated interactive code-to-architecture mapping recommendations," in *18th International Conference on Software Architecture*, 2021, pp. 173–183.

[14] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "GAER: Graph auto-encoders for unsupervised software architecture recovery," *Proceedings of the ACM on Software Engineering*, vol. 3, no. FSE, Jul. 2026.

[15] ——, "Graph representation learning for software architecture recovery." in *ESANN*, 2026.

[16] D. Amalfitano, M. De Luca, T. Santilli, P. Pelliccione, and A. R. Fasolino, "Automated software architecture design recovery from source code using llms," in *European Conference on Software Architecture*. Springer, 2025, pp. 73–89.

[17] R. Gupta, A. Jain, A. Gonzalez, A. Novikov, P.-S. Huang, M. Balog, M. Eisenberger, S. Shirobokov, N. Vũ, M. Dixon *et al.*, "Archagent: Agentic ai-driven computer architecture discovery," *arXiv preprint arXiv:2602.22425*, 2026.

[18] D. Fuchß, H. Liu, T. Hey, J. Keim, and A. Koziolek, "Exarch: Enabling architecture traceability by llm-based architecture component name extraction," in *SE2026*. Gesellschaft für Informatik, Bonn, 2026.

[19] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "Graph convolution networks for mapping source code entities to architectural modules," in *2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2025, pp. 472–481.

[20] V. Tzerpos and R. C. Holt, "The orphan adoption problem in architecture maintenance," in *Working Conference on Reverse Engineering*. IEEE, 1997, pp. 76–82.

[21] A. Florean, L. Jalal, Z. T. Sinkala, and S. Herold, "A comparison of machine learning-based text classifiers for mapping source code to architectural modules," in *15th European Conference on Software Architecture*, 2021.

[22] S. A. Rukmono, L. Ochoa, and M. Chaudron, "Deductive software architecture recovery via chain-of-thought prompting," in *44th International Conference on Software Engineering: New Ideas and Emerging Results*, 2024, pp. 92–96.

[23] N. Johansson, M. Caporuscio, and T. Olsson, "Mapping source code to software architecture by leveraging large language models," in *European Conference on Software Architecture*. Springer, 2024, pp. 133–149.

[24] N. Johansson and M. Caporuscio, "Mapping source code to software architecture using embedding models: Insights from an analysis of data properties," *SSRN Electronic Journal*, 2025, preprint, SSRN 5798927. [Online]. Available: https://ssrn.com/abstract=5798927

[25] J. Chen, Z. Wang, Y. Jiang, J. Pang, T. Zhang, M. Pan, and J. Sun, "NEGAR: network embedding guided architecture recovery for software systems," in *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2022, pp. 367–376.

[26] Y. Zhang, Z. Xu, C. Liu, H. Chen, J. Sun, D. Qiu, and Y. Liu, "Software architecture recovery with information fusion," in *Proceedings of the 31st ACM Joint ESEC/FSE*, 2023, pp. 1535–1547.

[27] R. Jabeen, M. Ericsson, J. Nordqvist, and A. Wingkvist, "Graph embeddings for software architecture recovery," *Nordic Machine Intelligence*, 2026. [Online]. Available: https://journals.uio.no/NMI/article/view/13153

[28] I. Stavropoulou, M. Grigoriou, and K. Kontogiannis, "Case study on which relations to use for clustering-based software architecture recovery," *Empirical Software Engineering*, vol. 22, no. 4, pp. 1717–1762, 2017.

[29] H. V. Nguyen, "Depends: Multilingual dependency analysis tool," https://github.com/multilang-depends/depends, 2023, accessed: 2025-08-17.

[30] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," *arXiv preprint arXiv:1710.10903*, 2017.

[31] J. Brunet, R. A. Bittencourt, D. Serey, and J. Figueiredo, "On the evolutionary nature of architectural violations," in *2012 19th Working Conference on Reverse Engineering*. IEEE, 2012, pp. 257–266.

[32] J. Lenhard, M. Blom, and S. Herold, "Exploring the suitability of source code metrics for indicating architectural inconsistencies," *Software Quality Journal*, vol. 27, no. 1, pp. 241–274, 2019.

[33] T. Lutellier, D. Chollak, J. Garcia, L. Tan, D. Rayside, N. Medvidović, and R. Kroeger, "Measuring the impact of code dependencies on software architecture recovery techniques," *IEEE Transactions on Software Engineering*, vol. 44, no. 2, pp. 159–181, 2017.

[34] M. P. Fay and M. A. Proschan, "Wilcoxon-mann-whitney or t-test? on assumptions for hypothesis tests and multiple interpretations of decision rules," *Statistics Surveys*, vol. 4, p. 1, 2010.

[35] N. Cliff, "Dominance statistics: Ordinal analyses to answer ordinal questions." *Psychological bulletin*, vol. 114, no. 3, p. 494, 1993.

[36] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," *Advances in neural information processing systems*, vol. 33, pp. 7793–7804, 2020.

[37] Q. Li, Z. Han, and X.-M. Wu, "Deeper insights into graph convolutional networks for semi-supervised learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.